Neural Computing and Applications (2025) 37:4035–4052
https://doi.org/10.1007/s00521-024-10323-x

ORIGINAL ARTICLE

Automatically weighted focal loss for imbalance learning

Nasibeh Mahmoodi[1] · Hossein Shirazi[1] · Mohammad Fakhredanesh[1] · Koroush DadashtabarAhmadi[1]

Received: 23 August 2022 / Accepted: 29 July 2024 / Published online: 19 December 2024

Abstract
In the context of class-imbalanced learning, most CNN-based classification algorithms encounter the problem of majority class gradient dominance, which makes them susceptible to bias toward the majority class. It is the consequence of the underlying assumption that the costs of misclassification are equivalent and that the class distribution is reasonably balanced. A standard way for dealing with the data imbalance is by using a loss function that favors the minority class. A constant weight is considered for each class in the loss function, which is generally inversely related to the number of instances in the class. In this paper, for multi-class imbalanced learning, we proposed Dynamic Weighted Focal Loss (DWFL), which determined the weight of each class for the next epoch automatically based on the classifier's results in all previous epochs. In this way, the better-learned class weighed less, and the less-learned class weighed more. Additionally, the application of focus loss reduced the importance of simple samples while increasing the importance of hard ones compared to cross-entropy. The experiments on several imbalanced datasets revealed that the performance of our suggested strategy (DWFL) is superior to that of previous resampling or reweighting methods with imbalanced classes. The experimental findings demonstrate the efficacy and convergence of the proposed methodology.

Keywords Imbalanced learning · Focal loss · Weighted loss function · Deep learning

1 Introduction

Many real-world domains, such as surveillance, health, and banking often contain imbalanced-class datasets. Classification of such datasets has been considered by machine learning researchers in recent years. A surveillance camera that is used to monitor public and private spaces and to identify people should check the videos to see if a street fight has occurred. This is an imbalanced learning problem in which most examples fall into the negative class, as there are very few images in which conflict has occurred. The same is true for recognizing natural disasters in the video. In the case of diagnosing dangerous diseases from medical images, although the positive class is more important, there is insufficient data available for learning the positive class. The term "class imbalance problems" (also known as "between class imbalance") refers to situations when there are less samples in one class than in the other. Standard classifiers typically fall into the majority class category. In the context of learning-related issues, imbalances in the distribution of the data can occur either between classes (Between-class imbalance) or even within a single class, which is referred to as within-class imbalance.

Deep neural networks (DNNs) have made significant advances in machine learning for a variety of classification tasks, including image classification, action detection, and speech recognition, with the help of novel concepts, new algorithms, novel network topologies, and training on large-scale datasets. The standard DNN training is still more challenging to generalize in the case of imbalanced datasets [1–3]. In the 1990s, Anand et al. [1] studied the backpropagation technique when data is imbalanced in shallow neural networks. They have demonstrated that in a

✉ Hossein Shirazi
hossein_shirazi@yahoo.com

Nasibeh Mahmoodi
Mahmoodi.nm@mut.ac.ir

Mohammad Fakhredanesh
fakhredanesh@mut.ac.ir

Koroush DadashtabarAhmadi
dadashtabar@mut.ac.ir

[1] Faculty of Electrical and Computer, Malek Ashtar University of Technology, Tehran, Iran

class-imbalanced scenario, the length of gradient components associated with a minority class is significantly smaller than the length of gradient components associated with a majority class. In other words, the computed net error gradient vector is dominated by the majority class to the point where the net error for the minority class samples is ignored. This quickly reduces the majority class error in the initial iterations, but it frequently raises the minority class error and slows convergence.

There are two strategies to overcome the class imbalance problem; algorithmic and data methods, which artificially rebalance the training objective. The algorithmic method is used to alter the classification costs or model structure to improve outcomes that benefit the minority class. Instead, changing data (reducing or increasing samples) is known as the data method. In [2], Johnson and Khoshgoftaar provided a good survey on various forms of DNN-based imbalanced learning in detail. Studying is suggested for more familiarity with the multiple methods presented in algorithmic and data methods.

A common data method of solving the imbalance issue is to resample, either by under-sampling such as (Random Under-Sampling) RUS, or by oversampling such as (Random Over-Sampling) ROS [3–5]. The disadvantage of oversampling is that it tends to produce errors because of noise or overfitting, whereas undersampling decreases the number of data available to train the model [6]. Pouyanfar et al. [7] suggested a model that simulates how people learn by placing more emphasis on notions that have not been learned before. Such models consider equal data from each class at the beginning of the training. The number of examples from each class is then increased or decreased according to the classification result in the previous epoch, however; it is challenging to determine the adequate initial number for each class. Also, if the model has a good result on the majority class in each epoch, the number of its samples will not change or even decrease during training. Therefore, many examples from the majority class that can contribute to better training do not contribute to training at all. We allow all samples to participate in the training by introducing dynamic weighting. By selecting the focal loss function, the easy samples of the majority class are automatically down-weighed.

In standard learning algorithms, the samples' weight and the concepts' weight are treated as constants during training, regardless of how much each is learned (samples and concepts), particularly in the case of class imbalances. The smaller classes in some samples have less contribution in the final loss value (total or average of each sample loss) and thus less contributions in updating the network weights.

There are two main issues with using a constant weight that is inversely proportional to the number of samples in each class $(\frac{\text{Total numberofdatapoints}}{\text{Numberofclasses}\times\text{Numberofsamplesinclass}})$. First, it treats all data points in the majority class equally, even though there can be an imbalance within the class. Second, the weight of the majority class significantly declines and eventually disappears when the sample sizes of the minority class are incredibly small in comparison to the majority. As a result, the informative data from the majority class have a very low impact on the training process, leading to a decrease in the overall performance of the classifier. For example, consider a scenario where the number of minority and majority class samples is 100 and 2000, respectively. The minority class would be given a weight of 10.5 $(\frac{2100}{2\times100})$, while the weight for the majority class would be 0.525 $(\frac{2100}{2\times2000})$. After normalization, these values become 1 and 0.05 for minority and majority classes, respectively. Consequently, the classifier is trained primarily on the limited data from the minority class, and the weight of the informative majority class data is extremely low. It leads to a decline in the classifier's overall performance. Utilizing the logarithm of weights is a solution. The logarithm of the two values are notably different from one another, however; tends to be quite close, which prevents it from accurately reflecting the significance of the minority class. In this study, the use of dynamic weighting is proposed, where the model is first trained with all the data and progressively gives higher weights to more challenging samples and classes as the training progresses. This approach ensures that the model learns from all the data and gives more emphasis to difficult samples and classes, leading to a more balanced and effective training process.

We suggest a new loss function, called Dynamic Weighted Focal Loss (DWFL), to pay greater attention to the examples and concepts (classes) that have not been fully learned throughout the training to improve the model's performance. Four improvements that are included in our DWFL loss function are as follows:

1.1 Dynamic weighting

Instead of applying a constant weight for cases throughout training, the suggested approach gives higher weights to concepts or classes that the model hasn't learned properly. We take into account the model's performance on previous training iterations when calculating the class weight. Since the results of CNN in the initial training epochs are not highly reliable and cannot be solely relied upon, it is recommended to use a moving average of the initial fixed weight and the weight obtained from the previous epoch. This approach helps to provide more stability and consistency in the training process.

1.2 Incorporating results from all previous epochs using an impact parameter, alpha

The weight of each class is established using the results of all prior epochs. Indeed, a class with poor model performance throughout several training iterations is given more weight in the following epoch. The findings from the current epoch are multiplied for this purpose by the impact factor alpha, while the results from the previous epoch are multiplied by 1-alpha. Due to the fact that the weights from the initial epochs are multiplied by (1-alpha) at each iteration, their influence diminishes over time. As a result, the impact of the initial epochs gradually decreases, and the model becomes more responsive to the recent training epochs.

1.3 Addressing both the within-class and between-class imbalance

In addition to addressing the between-class imbalance, our approach tackles within-class imbalance, as well. By using the focal loss function, we alleviate the problem of within-class imbalance. This means that during training iterations, a sample is given more consideration if it does not belong to the minority class, but the model fails to forecast it correctly.

1.4 Bias adjustment in the last layer

The proposed system gives a higher priority to the categories with more errors from the previous epoch. In the final layer of the network, we employ a bias correction strategy to increase the significance of the minority class. According to the number of samples in each class, we change the bias so that the minority class's inaccuracy rises in the early epochs.

The rest of paper is arranged as follows: Sect. 2 discusses focal loss. Section 3 reviews the related works. In Sect. 4, the principle of the proposed method is discussed. Section 5 discusses the experimental setup and datasets used as well as the results in each dataset in detail. Finally, Sect. 6 includes a conclusion.

2 Loss functions for imbalance learning

The most widely used algorithmic technique entails designing a new loss function such that the minority class has a higher misclassification cost compared to the majority class. Mean false error (MFE) and mean squared false error (MSFE) loss [7], balanced cross-entropy, and focal loss [8] that were recently introduced are the most popular and useful loss functions to deal with data imbalances.

Wang et al. [7] demonstrated that mean squared error (MSE) loss function fails to reflect the minority class errors when there is a high-class imbalance. They proposed MFE and MSFE loss functions that are more influenced by the errors from the minority class loss function. MFE (Eq. (3)) is made by combining mean false-positive error (FPE) and mean false-negative error (FNE). MSFE loss (Eq. (4)) was also introduced as an enhancement to the MFE loss, claiming that the error rates in the positive and negative classes will be more balanced.

$$\mathrm{FPE} = \frac{1}{N}\sum_{i=1\cdots N}\frac{1}{2}\left(d^{(i)} - y^{(i)}\right)^2 \tag{1}$$

$$\mathrm{FNE} = \frac{1}{P}\sum_{i=1\cdots P}\frac{1}{2}\left(d^{(i)} - y^{(i)}\right)^2 \tag{2}$$

$$\mathrm{MFE} = \mathrm{FPE} + \mathrm{FNE} \tag{3}$$

$$\mathrm{MSFE} = \mathrm{FPE}^2 + \mathrm{FNE}^2 \tag{4}$$

A common way to overcome class imbalance is to use a weighted cross-entropy loss function called balanced cross-entropy as shown in Eq. (5).

$$\text{balanced - CE}(p_t) = -w_t \log(p_t) \tag{5}$$

Weighting factor w_t is inversely related to the class frequency, and p_t is the model predicate probability of the sample true label. For multi-class problems, Eq. (5) is rewritten as:

$$\text{Multi - class - balanced - CE}(p_t) = -\sum_{1}^{C} w_i t_i \log(p_i) \tag{6}$$

where C determines the class number in the data set, w_i is the ith class weight, t_i is the ith item of the true label vector for the input sample, and p_i is the ith item of the predicted probability vector model for the ith input sample. It is difficult to determine the most appropriate weighting value (w_i) for each class. Therefore, we propose a dynamic weighted loss function in this paper that calculates the weight of each class according to the results of the previous iteration of training. When using imbalanced binary classification, the model emphasizes learning from the difficult negative cases and introduces a focal loss based on standard cross-entropy to down-weigh the well-trained samples. In this study, we adopt focal loss to the multi-class classification problem.

Lin et al. [8] introduced the concept of focal loss (FL) in their one-stage model, RetinaNet, which tackles the severe class imbalance. This imbalance frequently occurs in object detection issues, where negative background samples are far more than positive foreground samples. The FL

decreases the effect of easily classified examples on the loss by reshaping the Cross-Entropy (CE) loss as Eq. (7).

$$\mathrm{FL}(p_t) = -\alpha_t(1-p_t)^{\gamma}\log(p_t) \tag{7}$$

To make a Focal Loss, modulating factor $\alpha_t(1-p_t)^{\gamma}$ is multiplied by cross-entropy. Hyperparameter $\gamma \geq 0$ controls how much easy examples are down-weighted, and the minority class is given more importance for a binary classification using $\alpha_t \geq 0$, which is a class-wise weight. In addition to class imbalance problems, the FL method is also suited to hard sample problems (hard samples are examples or instances where the model's predictions do not show significant improvement even as the training progresses). The majority and minority class gradient problem described by Anand et al. [1] is addressed by allowing the minority class to make a more significant contribution to weight updates and preventing the majority class from the dominating loss. It has minimal impact on training time and can be easily integrated into existing models.

Due to the extreme imbalance, training RetinaNet with standard CE loss diverges and fails quickly, however; the Average Precision (AP) of the model improves significantly to 30.2 when the bias in the last layer of the model is initialized by $b = -\log((1-\pi)/\pi$. This makes the prior probability of an object detecting $\pi = 0.01$. On the other hand, "at the start of training, every anchor should be labeled as foreground with the confidence of $\sim \pi$" [8]. Based on the results of the experiments, $\gamma = 2.0$ and $\alpha_t = 0.25$ are selected as appropriate hyperparameters.

Several state-of-the-art ones-stage and two-stage detectors are compared to the RetinaNet using the COCO dataset [9] (COCO is a dataset used in the field of computer vision for object detection and image captioning tasks. It consists of over 200,000 labeled images with annotations for objects). In terms of AP gains, it outperformed the Faster R-CNN [10] and DSSD513 [11] (the best two-stage detector and the next-best one-stage detector) by 4.0 and 7.6 points, respectively. They initialize the last layer bias to $b = -\log((1-\pi)/\pi)$, where π indicates that an anchor with the confidence of $\sim\pi$ should be labeled as foreground at the start of training. As a result, if there are many background anchors, a high destabilized loss value is generated in the initial training iteration.

3 Related works

Few works have been done so far on FL performance in image classification, and its effectiveness has not been studied yet in detail. In the following, we go through some of the research that has been done in the area of FL. Nemoto et al. [15] used focus loss to identify a multi-class problem known as rare building change. The training data included 203,358 total images with high imbalanced ratios of 900. There have been six classes, including rebuilding, demolishing, repainting roofs, and laying solar panels. 200,000 of the 203,358 total images were included in the negative class (i.e., no change), and the remaining images belonged to the other five classes. Class-wise accuracy was utilized to compare FL and CE loss on the validation set. To better comprehend the impact of γ, the value of the hyperparameter γ in modulating factor was adjusted in the range $\gamma \in [0, 5]$.

The result shows FL prevents overfitting by increasing iterations. Increasing γ causes the total loss to decrease quickly and the model to overfit more slowly. The solution used by Nemat et al. [15] to overcome the data imbalance is data augmentation, not Focal Loss and, the Focal Loss function is just used to eliminate over-fitting. On the other hand, FL had higher precision in just one of the three chosen classes (repaint roof, no-change, and laying solar panel). In contrast, Ce (= 0) had better accuracy than FL in two other classes (no-change and laying solar panel), therefore; it cannot be explicitly claimed that the Focal Loss function has improved the outcome in this instance.

The FL-based Neural Network Ensemble (FLANNEL) was proposed by Zhi et al. [12] for COVID-19 detection. It is a weighted ensemble of five pre-trained networks (Densenet161, InceptionV3, ResNeXt101, Resnet152, and Vgg19_bn), which are fine-tuned by their COVID-19 database. This can be considered a multi-class classification task since they must be distinguished among chest X-ray images from COVID-19 patients, other pneumonia patients, and healthy patients. Using a neural network to learn the weights of each base model, they combined the output of the models to determine a final classification based on a weighted ensemble. The binary focal loss [8] function was adapted for multi-class classification tasks to overcome the challenge of class imbalance in this issue. They compared their results to five individual models and two common ensemble tactics, voting and stacking (one-layer and two-layer MLP. With an F1-score of 0.8168, FLANNEL outscored other approaches in the combination of the five chosen base learners. The multi-class standard cross-entropy loss was employed in place of FL to create a CE version of FLANNEL, and resampling techniques (ROS and RUS) were applied on top of it to assess the effectiveness of focal loss to manage imbalance learning. FLANNEL beat them by more.

The use of Focal Loss in successful generative networks such as Generative Adversarial Network (GAN) has also recently come to the attention of machine learning researchers. Fei et al. [13] introduced focal-GAN for supervised and unsupervised image generation and image-to-image translation. They proposed a version of FL named incremental focal loss (IFL). IFL allows training to

gradually place a more prominent emphasis on hard examples rather than easy examples that would overwhelm the generator and discriminator. A mechanism called enhanced self-attention (ESA) was also proposed to boost the generator's powers of representation. To demonstrate efficiency for different tasks, they applied their proposed method to various GANs, such as Pix2Pix [14], SRGAN [15], and SAGAN [16] as the baseline networks. They used the diverse dataset in different tasks, including face photo-sketch synthesis, map ↔ aerial-photo translation, super-resolution reconstruction, and image generation on CelebA [17], LSUN [18], and CIFAR-10 [19]. Compared to existing loss functions (CE loss [20], LS loss [21], and Hinge loss [22], IFL boosted the learning of GANs. It also showed intuitive comparisons that in all of these tasks, focal-GAN can create a quality realistic image.

Mukhoti et al. [23] proposed an approach for calibrating DNNs based on focal loss. They demonstrated that the application of focal loss for calibration allows us to discover models that are already well-calibrated, as opposed to cross-entropy loss. They studied the properties of focal loss as an alternative loss function that achieves better accuracy and calibration of classification networks than cross-entropy loss. The authors provided a thorough analysis of the factors that lead to miscalibration and used this information to justify the excellent empirical performance of focal loss models. Specifically, they demonstrated that focal loss maximizes entropy and minimizes KL divergence between predicted and target distributions. Because its design regulates the weights of a network during training, reducing overfitting caused by NLLs, and improving calibration. In almost all cases, they achieved state-of-the-art calibration without compromising accuracy by experimenting on a variety of datasets and with a variety of network architectures such as ResNet, DenseNet, treeLSTM, and others.

4 Proposed method: dynamic weighted focal loss (DWFL)

There are two issues in the context of class-imbalanced: First, "hard concepts" (classes) refer to classes that are difficult for the model to learn well, such as the minority class, which the model cannot learn it due to the small sample size. Second, for hard samples, whether they belong to the majority or minority class, the model prediction does not become better as training progresses.

4.1 Dynamic weight

Dynamic weight is introduced in this research to address the first issue (hard concept mining), as mentioned in the same section, and focused loss [11], has been used to address the second issue (had sample mining). The authors in [8] incorporated the modulating factor $(1 - p_t)^{\gamma}$ to cross-entropy loss. This modulating factor reduces the weight of the well-classified samples (easy samples) in the final loss function. Tunable focusing parameter γ increases the effect of modulating factor. Further details are provided in Sect. 2. Between-class imbalance can lead to the emergence of hard samples and hard concepts for the model. Within-class imbalance can also contribute to making examples hard. Addressing both hard classes and hard samples can help alleviate the issues caused by both types of imbalances (between-class and within-class imbalances).

In this section, by introducing the Dynamic Weighted Focal Loss function (DWFL) (Eq. (10)), we intend to guide the learning process in the training phase in such a way that it focuses more on concepts that have not been learned well so far. We achieve this by defining dynamic weights. To determine how well the model has learned each class (or concept), we use the results of the classifier from previous training epochs. We analyze the validation data (the percentage of data that is randomly selected) and results of the model that has been trained so far to measure the performance of the classifier in each class (performance can be measured using per-class accuracy or per-class F1-score metrics). Therefore, instead of considering the weight of the classes, constant from the beginning to the end of the training, we define the weight of the class in the inverse relationship with its performance in all previous epochs. If ϕ^e represents the performance of classes at the end of epoch e, the class weights (w^e) equal $1 - \phi^e$. This can be used as the weight of the classes in calculating the loss function in the next epoch, represented by the Eq. (8), which is a number between 0 and 1. While this vector can be used for class weights in the next epoch, we improve it by incorporating the results of all previous epochs.

In fact, we employ this technique to identify concepts that the model finds challenging to learn. They are known as "hard concepts", so; we try to assign them higher - weight. More specifically, a hard concept is one for which the model's performance has not considerably improved over the course of the training and which has not been learned successfully throughout the prior training epochs. Therefore, relying solely on the results of a single previous epoch cannot fully demonstrate the level of difficulty of a concept. To address this issue, we use the results of all previous epochs to calculate the weight of a concept (or its level of difficulty). We also use the hyperparameter alpha to determine the impact of epochs. Assume that w is the weight of the class up to the previous epoch $(e - 1)$, and w^e is the weight derived from the results of the latest epoch. We use the weighted sum of these two weights to calculate

the weight of the class in the next epoch ($e + 1$). The weight w^e contributes to determining the difficulty level of a concept with the coefficient alpha, whereas the weight w contributes with the coefficient 1-alpha. Consequently, the weight that is used in the next epoch is obtained by adding these two weights with their respective coefficients (Eq. (8)). The steps of the process are illustrated in Fig. 1.

All dataset samples used to train the CNN are initially weighted equally for each class ($w = 1$). The weight of each class is updated as training progresses in accordance with the results of the prior training interaction. Suppose ϕ^e is a c-element vector in epoch e that demonstrate to us how much each class is learned (precisions or F1-scores for classes) in the previous iteration (Eqs. 8 and 9).

$$w^e = \frac{1 - \phi^e}{\max(1 - \phi^e)} \quad \text{for each class } i.\, 0 \leq \phi_i^e \leq 1 \tag{8}$$

$$w = \alpha w^e + (1 - \alpha) w \tag{9}$$

For example, for 3 classes and after epoch 1, if ϕ^2 = F1-Score = [0.4, 0.7, 0.8] then W^2 = [1, 0.5, 0.33]. Therefore, according to the weights obtained in the example, we see that the class with the lowest F1-score (0.4) has the highest weight (1), and the class with the highest F1-score (0.8) has the lowest weight (0.33). Then, the weight w^e is added to the previous weight vector (w), multiplied by the 'α' to determine the weight vector (w) for use in the loss function in the next epoch. A detailed algorithm of our proposed method is provided in Table 1.

4.2 Multi-class dynamic weighted focal loss

We adapt focal loss and its modulating factor for multi-class problems. Our dynamic weighted focal loss is shown in Eq. (10).

$$\text{DynamicWeightedFocalLoss(DWFL)} = \sum_{c=1}^{C} w_c^e (1 - p_c^i)^{\gamma} t_c^i \log p_c^i \tag{10}$$

where w^e represents the weight calculated up to eth epoch, and w_c^e is the cth element of it, p^i is the model predicted probability for ith sample, which is obtained by applying SoftMax to the top of the last layer of CNN network, and $t^i \in \{0.1\}^C$ is the ground truth vector for the ith sample. As shown in Eqs. (11) and (12), the regression model's output is turned into a class probability distribution using the softmax activation function. We add a SoftMax layer on top of CNN's output layer.

$$s = \theta^{\circ} \left(F^{\circ -1}\right)^T + b^{\circ} \tag{11}$$

$$p = \text{softmax}(s) \tag{12}$$

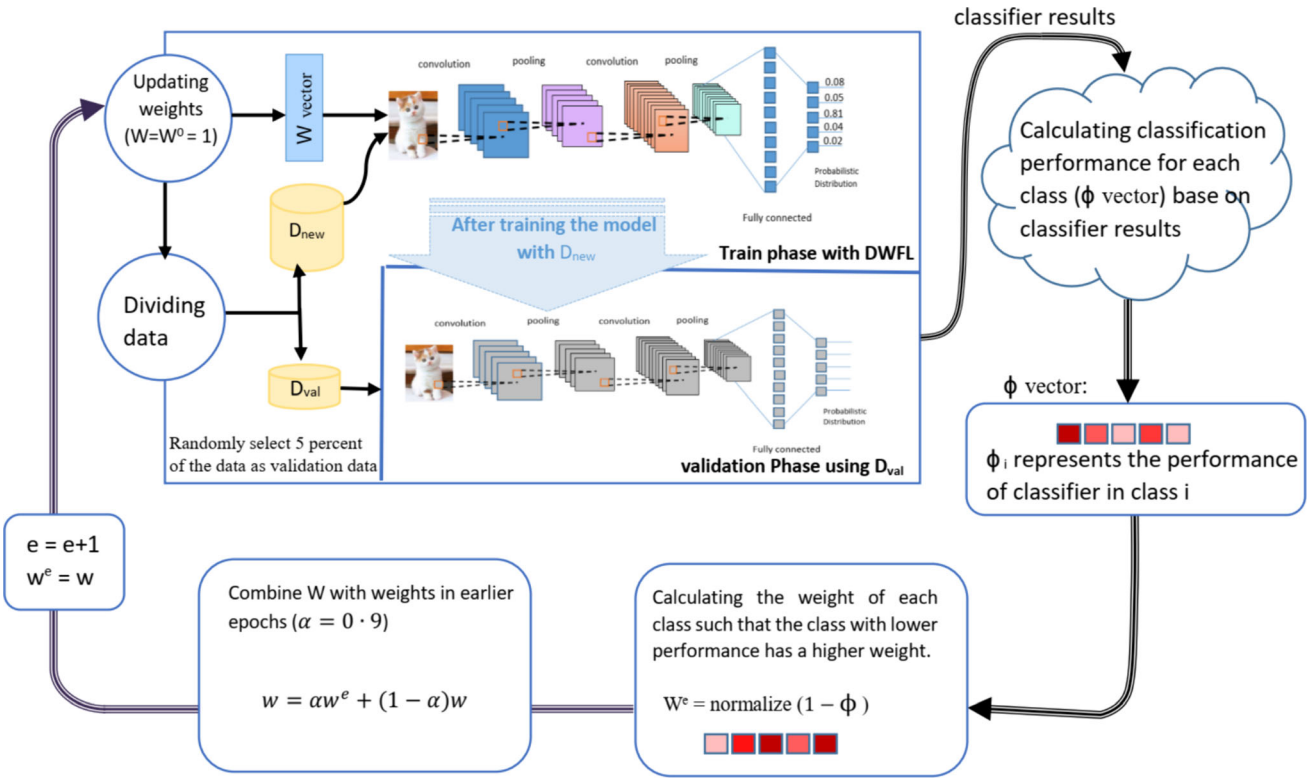

Fig. 1 The steps of proposed model in each training epoch

Table 1 The proposed algorithm for training CNN with dynamic weighted focal loss

Input: Training data D, initial model M Output: Trained classifier M compatible with imbalanced data
1. Initialize the weight of each class, $w_i = 1$ where i=1,…, C and (the dataset has a total of C classes) **2.** Initialize the bias of the last layer based on dataset properties. **3.** For e=0 to the number of epochs: **a.** Set $w^e = w$ **b.** Randomly select 5% of D as validation data D_{val}. **c.** Create new training data D_{new} by removing Dval from D. **d.** Train model M using D_{new} and update the model weights θ using dynamic weighted focal loss: - $dwfl = \frac{1}{\lvert Dnew \rvert}\sum_{(d_i.y_i)\in D_{new}} \sum_{c=1}^{C} w_c^e (1-p_c^i)^\gamma t_c^i log p_c^i$ where p^i is the predicted probability vector using SoftMax on model M for d_i **e.** Evaluate the model M on the validation data D_{val} - Calculate the performance vector ϕ^e, where ϕ_i^e represents the performance of the i-th class, **f.** Calculate the weight vector for the current epoch, w^e = normalize $(1-\phi^e)$ **g.** Combine the newly calculated weight with the weight from previous epochs: - $w = \alpha w^e + (1-\alpha)w$

where s is the score of the model for the input sample, θ° is the output layer weight matrix that determines how the output of the last fully-connected hidden layer is connected to the output layer, $F^{\circ-1}$ is the feature map matrix that comes from the previous layer, and b° is the bias matrix in the layer of output. Equation (10) calculates the loss value of a single sample, and to compute the overall loss for an epoch of samples during backpropagation, the average of the individual losses is utilized (

$$dwfl = \frac{1}{|Dnew|}\sum_{(d_i.y_i)\in D_{new}} \sum_{c=1}^{C} w_c^e (1-p_c^i)^\gamma t_c^i \log p_c^i$$

).

4.3 Change in last layer bias

Strategy is to alter the last layer of bias, which causes the predicted probability of minority classes to fall and the misclassification of minority classes to rise at the beginning of the train. The number of bias vector elements in the last layer equals the network output, i.e., the number of classes. We select a bias for each class so that for the class with a smaller number of samples, softmax produces the lower predicted probability. We select the value of the bias vector using cross-validation. For example, for a 3-class problem with 800, 500, and 1000 samples in each class, $b^\circ = [-0.5, -1.0, 0.0]$ can be a choice for each class, respectively, that produces a prior probability equal to $p = [0.3072, 0.1863, 0.50]$ (-0.5 is inverse softmax of 0.3072). The bias is selected so that the minimum value of predicted probability belongs to the class with the least number of samples. Changing the bias works in the same way as changing the threshold and greatly affects tending education toward the minority class. We know that lower predicted probability leads to higher loss. One theory is to manually decrease the predicted probability of minority class samples, which increases the penalty for minority class samples and minority class contributions to overall loss. To achieve this, especially in the initial iterations, we use the bias in the last layer favoring the majority class, which causes more penalties for the minority class and consequently more attention to the minority sample. By choosing a bias that reduces the predicted probability of the minority class, we can increase the minority class contribution to the final loss. To achieve this, especially in the initial iterations, we use the bias in the last layer, which favors the majority class. This bias results in more penalties for the minority class and, as a result, more focus on the minority sample. We can raise the minority class contribution to the ultimate loss by selecting a bias that lowers the expected probability of the minority class.

At the end of each epoch, the final focal loss is calculated as the average focal loss of the predicted probability for all samples.

4.4 Ensemble

Our proposed method performs well on the minority classes, however; if we want to improve overall performance, we can use a voting scheme across the proposed classifier and a classifier that has not used weighing in favor of the minority class during training. By combining the predictions of these classifiers, we can achieve a classifier that has good performance on both the minority and majority classes. Suppose that our proposed model is M and consider a model called M' that has the same configuration as our CNN, but uses the CE loss function rather than the DWFL and leaves the bias of the last layer unchanged. If we train M and M' on the same database and compare the

results, we see that M had better accuracy on classes with fewer numbers, and M' had better accuracy on classes with more data. For perfect performance in all classes, we ensemble M and M' in the testing stage using the weighted ensemble method. As depicted Fig. 2, for each new sample, the predictions of classifier M (ρ) and classifier M' (ρ') are combined using the weighted ensemble method. The combination is based on the weights δ and δ', which are calculated for ρ and ρ', respectively. The weight δ is determined by the equation $\delta_i = \frac{\sum_{i=1}^{c} n_i}{c \times n_i}$ for i = 1 to C, and δ' is calculated as $1 - \delta$.

By this approach, classifier M plays a more prominent role in classifying smaller classes (in the sense of number), while classifier M' takes on a greater role in classifying larger classes. To show the efficiency, in the next section, we examine our proposed method on four different datasets, including the Synthesis dataset [24], Cifar-10 [19] dataset, EMNIST by-merge dataset [25], and Actitracker dataset [26].

5 Experiments and results

The performance of our proposed method was evaluated by comparing to the benchmark models employed to four datasets; a Synthetic dataset [24], Cifar-10 [19], EMNIST by_merge [25], and Actitracker [26]. The proposed CNN for categorizing the Synthesis dataset is discussed in Sect. 5.1, and the explanation of the Cifar-10 dataset and its related CNN is provided in Sect. 5.2. The Cifar-10 dataset is not naturally imbalanced, hence; to create the imbalanced variants of Cifar, we randomly selected a different number of examples from each class. Further, Actitracker and EMNIST-by merge are class-imbalanced datasets. For each dataset, a CNN network is suggested to be compatible with the database. For more complex datasets, CNN is more complex and for a simpler dataset, a simpler network with fewer layers is recommended.

We initially calculated the accuracy in each class of the dataset to investigate the impact of the algorithm on the performance of the classifier separately on minority and majority classes. Then, we used accuracy, balanced accuracy, and F1-score metrics to assess the overall performance of the proposed method and compared it with other strategies. Moreover, to investigate the performance of our proposed method with increasing class imbalance, we employed ROC AUC as the evaluation metric. These metrics are described below.

All metrics are computed based on the confusion matrix (Fig. 3), where TP and TN refer to the number of samples that are correctly predicted in the positive and negative classes, respectively, while FN and FP refer to the number of samples that are incorrectly predicted as negative and positive, respectively.

$$\text{True positive rate (TPR)} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{13}$$

- Recall
- Sensitivity

$$\text{True Negative Rate (TNR)} = \frac{\text{TN}}{\text{TN} + \text{FP}} \tag{14}$$

- Specificity

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{15}$$

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \tag{16}$$

$$\text{Balanced Accuracy} = \frac{\text{TPR} + \text{TNR}}{2} \tag{17}$$

$$\text{F1} - \text{score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{18}$$

TPR (Eq. (13)), also known as Recall or Sensitivity, is actually the classifier's accuracy in the positive class and TNR (Eq. (14)), also known as Specificity, is the classifier's accuracy in the negative class. Precision (Eq. (15)) measures the proportion of true positive predictions out of all the positive predictions made by the classifier. In other words, precision represents the accuracy of the positive predictions. Accuracy (Eq. (16)) is a commonly used performance metric in classification that measures the overall correctness of the classifier's predictions. It represents the proportion of correct predictions (both true positive and true negative) made by the classifier out of all the predictions it has made and Recall (Eq. (13)), is the proportion of true positive predictions out of all the actual positive cases in the dataset. Balanced accuracy (Eq. (17)) is a performance metric that takes into account the imbalance between the number of positive and negative cases in the dataset. It is the average of sensitivity (TPR) and specificity (TNR) and provides a more accurate measure of classifier performance when the classes are imbalanced. Unlike accuracy (Eq. (16)), which can be misleading in imbalanced datasets, balanced accuracy ensures that the performance of the classifier is evaluated equally for both classes. This is particularly important when there is a significant difference in the number of samples between the two classes, as it may cause the classifier to be biased toward the majority class.

F1-score (Eq. (18)) combines precision and recall into a single score. It is the harmonic mean of precision and recall

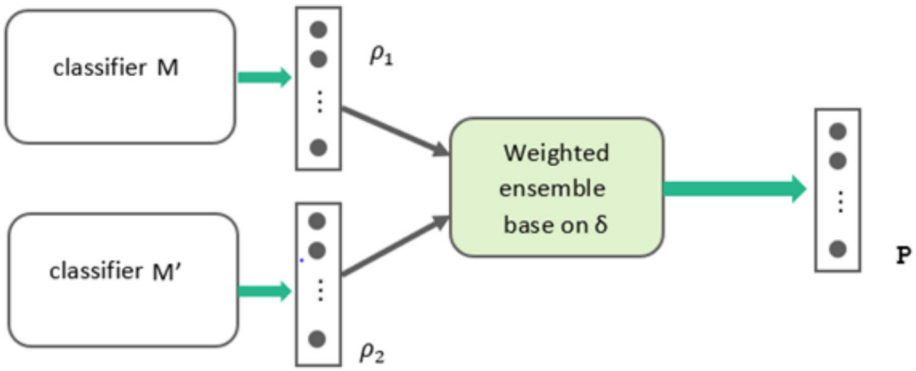

Fig. 2 Ensemble of two classifiers, M and M', combining their predictions for improved performance. The classifier M demonstrates better accuracy on minority class samples, while M' performs better on majority class samples

and provides a more balanced measure of classifier performance than either precision or recall alone and is less affected by the imbalance in the dataset. Precision is the proportion of true positive predictions out of all the positive predictions made by the classifier. F1-score ranges between 0 and 1, with a higher value indicating better classifier performance. A perfect classifier has an F1-score of 1, while a completely random classifier has an F1-score of 0.5.

Receiver Operating Characteristic (ROC) curve is a graphical representation of the performance of a classifier as the discrimination threshold is varied. The ROC curve is created by plotting the true positive rate (TPR) on the y-axis and the false positive rate (FPR) on the x-axis, as the classification threshold is varied. The area under the ROC curve (AUC) is a performance metric that measures the overall ability of the classifier to discriminate between positive and negative classes, regardless of the threshold used. A perfect classifier have an AUC of 1, while a completely random classifier have an AUC of 0.5. One of

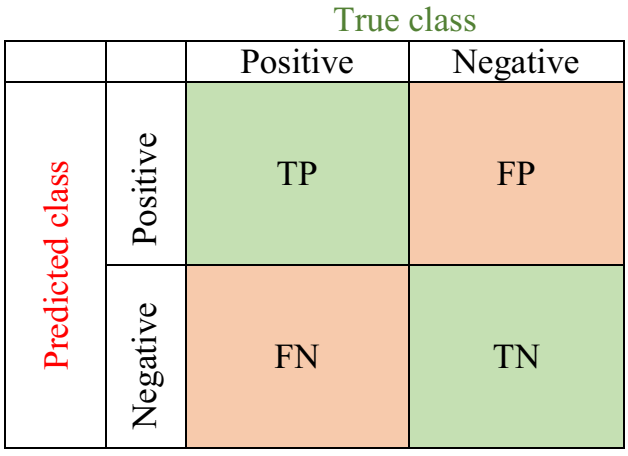

Fig. 3 Confusion matrix, TP is the true positive (the correctly predicted positive class outcome of the model), TN is the true negative (the correctly predicted negative class outcome of the model), FP presents false positive (the incorrectly predicted positive class outcome of the model), and FN defines false negative (the incorrectly predicted negative class outcome of the model)

the advantages of ROC AUC over other performance metrics, such as accuracy, precision, recall, and F1-score is that it provides a single measure of classifier performance that is insensitive to class imbalance and threshold selection. This makes it a useful metric for evaluating the performance of classifiers in a variety of settings.

5.1 Experiment on the synthetic dataset

For all training examples in three classes, each input $x \in R^{10}$ and a ten-dimensional standard Gaussian distribution were used to randomly generate 10 input variables. Briefly, the decision boundaries between classes were nested spheres that had the same center and were ten-dimensional. There were 2300 training observations in total. The number of training samples in c1, c2, and c3 was almost 800, 400, and 1000. Calculating the error rate was performed using 10,000 examples in the test set that are independently drawn. The CNN model configuration details for the synthetic dataset are provided in Table 2.

As shown in Table 2, the proposed CNN had six layers with a layer of soft-max activation at the end. The first layer was a 1D-convolution layer with 3×1 kernels and the second layer was a 1D-convolution layer with 2×1 kernels followed by a 2×1 max-pooling layer. The two layers had a filter map of sizes 32 and 64, respectively. After that, two fully connected layers had 128 and 16 neurons, respectively, and a dropout ratio of 0.2 was used in the 128 neurons fully-connected layer. All layers were activated using the ReLu function. Finally, the output layer, which was a fully-connected layer with three neurons was activated using the SoftMax function. Adagrad optimizer with learning rate = 0.3 and exponential scheduler with gamma = 0.9 was used. Weights and bias were used initially with the default PyTorch setting except for the last layer that bias = [− 0.4, − 0.6, 0].

The proposed CNN (represented in Table 2) was trained on the Synthetic dataset with three different loss functions, including the proposed Dynamic Weighted Focal Loss (DWFL), cross-entropy, and weighted class entropy. The

per-class accuracy for the 10,000 testing sample is shown in Fig. 4. The majority classes (C1, C3) and the minority class (C2) both benefit most from our suggested method (DWFL).

In Table 3, the proposed method is compared with common imbalance learning methods using measures, such as train accuracy, test accuracy, balanced accuracy, and F1-score. The evaluation of the data in the table shows that the dynamic weighted focal loss method is superior to other approaches for handling imbalanced data. When utilizing DWFL instead of standard cross-entropy loss, training accuracy was 3.83% greater than CNN with standard cross-entropy loss, while test accuracy, balanced accuracy, and F1-score were 2.96, 3.07, and 2.7% higher, respectively. It should be noted that the final layer in CNN used for methods 1–3 was modified in a way that increased the minority class error in the early epochs. As a result, it performed better than using the default initializing PyTorch values for the network. If we used the default values, the results of these methods were lower than the values in the table.

5.2 Comparison of DWFL with balanced cross-entropy (BCE)

Comparison of DWFL with balanced cross-entropy (BCE) balanced cross-entropy [27, 28], also called weighted cross-entropy, is the common solution to deal with class-imbalanced data. The loss function shown in Eq. (2) was used as the convolutional neural network loss function, and the epoch number is equal to DWFL epochs (15 epochs). Train and test accuracy, balanced accuracy, and F1-score of DWFL were, respectively, 2.32, 1.89, 1.88, and 1.73% higher than the test and train accuracy of BCE, and the BCE had better results than standard cross-entropy CNN (raw 1 of the table).

5.3 Comparison of DWFL with SMOTE method

The second most popular method for addressing data imbalances is the synthetic minority oversampling technique (SMOTE) [29, 30]. The result of SMOTE on Synthetic data is represented in row 3 of Table 3. Our approach outperformed SMOTE in terms of train accuracy, test accuracy, balanced accuracy, and F1-score. Although standard cross-entropy CNN's accuracy on test samples was improved by using SMOTE.

5.4 Comparison of DWFL with Adaboost-CNN method

We compared our method with a new CNN-based multi-class AdaBoost method introduced in [31] that used the result of previous CNN for weighing next CNN data samples. Ten CNN was used for the base estimator and SAMME [24], and the multi-class AdaBoost method was used for weighing samples and combining the result of CNNs. With higher percentages of 2.63, 2.06, and 1.98 for the train, test, and balanced accuracy, respectively, as well as a higher percentage of 2.02 for the F1-score, our strategy produced superior results. The CNN setup utilized in Adaboost was comparable to that described in [33].

5.5 Comparison of DWFL with residual network (ResNet) method

Since in various fields, the residual network [32] has state-of-the-art results [33], we trained ResNet with three residual blocks on Synthetic dataset, and it achieved 94.87% accuracy on the train-set and 82.81% on the test-set. As seen in Table 3, our strategy produced the best outcomes when compared to all of these methods.

5.6 Experiment on the Cifar-10 dataset

Cifar-10 [19] is a database consisting of 10 classes, including airplanes, cars, birds, cats, deer, dogs, frogs, horses, ships, and trucks. Each class contains 5000 training data and 1000 test data, and the entire database contains 60,000 images. The data size is about 160 MB. Each image in this dataset is 32×32 pixels. In Fig. 5, you can see an example of the images in this database.

Since the ResNet had state-of-the-art results on Cifar-10, to examine our method on the Cifar-10 dataset, we selected ResNet-18 as the base classifier. For the Cifar-10 experiment, we used ResNet-18, which had 91.47% accuracy on the manually imbalanced training dataset. We trained ResNet-18 from scratch to compare the results of the imbalanced dataset. Since the pre-learned weights might not accurately reflect the class imbalance effect, we did not rely on pre-trained models. The models were instead trained from scratch to simulate real-world situations where the data was imbalanced, and there was no pre-trained network for the beginning of the train. The grid search method was used to determine the appropriate hyperparameters for each competing method. To execute the proposed technique on the Cifar-10 dataset, we first imbalanced the training data by randomly reducing the number of five classes to 10, 20, 30, 40, and 50% of the current rate and reviewing the results. The results are shown in Table 4. On average, DWFL-ResNet had higher accuracy of 92.56% compared to the CE-ResNet with 91.47%. It also has higher test accuracy, balanced accuracy, and F1-score compared to other methods.

Figure 6 shows the testing accuracies for ResNet-18 with Cross-Entropy (CE) loss function, ResNet-18 with

Table 2 The topology of the proposed CNN for the Synthetic dataset

Layer	Number of filters/number of neurons	Kernel size
1D Convolution ReLU	32	3 × 1
1D Convolution ReLU	64	2 × 1
Max-Pooling kernel	–	2 × 1
Fully connected ReLU Dropout 20%	128 Neurons	–
Fully connected ReLU	16 Neurons	–
Fully connected SoftMax	3 Neurons	–

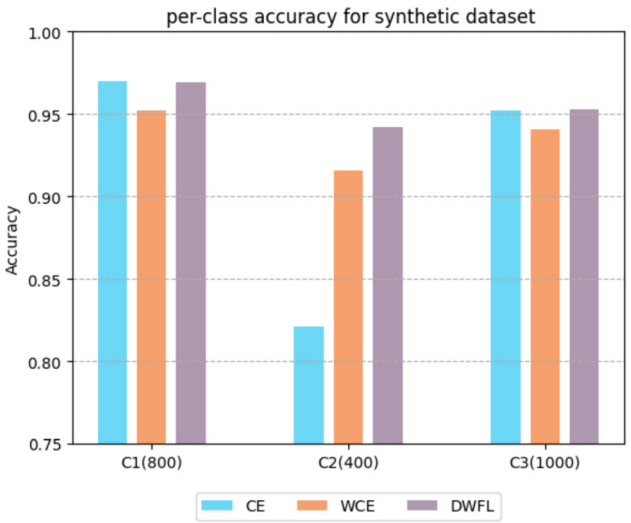

Fig. 4 Comparing testing accuracies for CNN with three loss functions: Cross Entropy (CE), Weighted Cross-Entropy (WCE), and our proposed Dynamic Weighted Focal Loss (DWFL) for different classes on the synthetic dataset

Weighted Cross-Entropy (WCE) loss function, and ResNet-18 with dynamic weighted focal loss for different classes on the manually imbalanced Cifar-10 dataset. Based on Fig. 6, it is evident that DWFL achieves higher accuracy in most classes, including both minority classes such as "cat" with 500 samples and classes like "ship" with 5000 samples.

5.7 Experiment on the EMNIST-by merge dataset

The EMNISTby-merge contains 47 classes [30]. Thirty-seven of the classes are letters, while ten are numerals. The EMNIST –by merge dataset combines uppercase and lowercase classes, except for letters with different representations in uppercase and lowercase (a, b, d, e, f, etc.). The black bars in Fig. 6 show how the EMNIST –by merge dataset was imbalanced and had a varying number of examples in each class. It had 814,255 samples in total, 731,668 samples for training, and 82,587 samples for testing. To facilitate a better comparison of the proposed method's performance on minority and majority classes, the data distribution in each class was presented along with the per-class accuracy of the methods in Fig. 7. Instead of displaying the number of samples in each class, normalized values were shown as black bars. For this purpose, the number of samples in each class was divided by the size of the largest class, and the resulting values were plotted as black bars I Fig. 7.

Since VGG5 [34] has the state-of-the-art result on the EMNIST letter [25] (that is a balanced database), we selected it as the base classifier for comparing CE and DWFL. To adapt the VGG5 to our database, we changed output neurons (number of class) in the last layer according to the EMNIST –by merge dataset and then trained VGG5 from scratch on EMNIST –by merge with two different loss functions of cross-entropy and dynamic weighted focal loss (gamma = 2). Test accuracy for each class is represented in Fig. 7. VGG5 + DWFL outperformed VGG5 in most classes.

Table 3 Comparing the results obtained by various methods on Synthetic data

Model name	Train accuracy%	Test accuracy%	Balanced_accuracy%	F1-score%
Cross-entropy-CNN	94.43	93.18	92.41	93.06
Balanced-cross-entropy-CNN	95.94	94.25	93.60	94.03
SMOTE	95.48	94.45	94.15	94.22
Adaboost-CNN	95.61	94.08	93.52	93.74
Cross-entropy ResNet	94.87	82.81	80.48	81.58
Dynamic weighted Focal Loss (gamma = 2)-CCN(proposed model)	**98.26**	**96.14**	**95.48**	**95.76**

The best result in each column is indicated in bold and underlined format

Table 5 demonstrates that the accuracy of VGG5 using our suggested loss function (DWFL) is greater than the accuracy of VGG5 when using the cross-entropy loss function with 1.09, 1.88, 2.05, and 1.47% higher train accuracy, test accuracy, balanced accuracy, and F1-score, respectively.

5.8 Experiment on the Actitracker dataset

We use the Actitracker [26] dataset, provided by Wireless Sensor Data Mining (WISDM) lab. It contains 36 users' data, which are collected using smartphones in their pockets at a sampling rate of 20 samples per second. While the user performs six different activities in a controlled

Fig. 5 Cifar-10 dataset [19]:7 random images for each of the 10 classes

environment, the data is collected and includes values for 3D acceleration along the x, y, and z axes. These activities include downstairs, jogging, upstairs, standing, sitting, and walking. As shown in Fig. 8 (black bars), the dataset was imbalanced concerning the performed activities (class labels). The black bars represent the normalized number of samples per class in Fig. 8. The proposed DWFL method demonstrates good performance both in minority classes like "standing" and in majority classes like "walking."

The proposed CNN with six layers is shown in Table 6, and a SoftMax function is placed on top of the CNN. Adam optimizer with a learning rate of 0.001 was used for the training of CNN. We trained CNN separately with three different loss functions; cross-entropy loss function, weighted cross-entropy, and dynamic weighted focal Loss Table 7. Comparison of the performance of Cross-Entropy (CE), Weighted Cross-Entropy (WCE), and our proposed DWFL on the Actitracker dataset. The results show the efficiency of the proposed method (DWFL). Considering test accuracy, with distances of 1.06 and 1.89%, it is better than CE and Weighted CE, respectively. According to balanced accuracy and F1-score, DWFL (gamma = 2) is superior by obtaining 89.65 and 88.23 points, respectively. Accuracy in each class is represented in Fig. 6, and DWFL has the best result in all classes, especially in minority classes (standing, sitting).

5.9 Investigating the effect of increasing the imbalanced ratio on the performance of DWFL

The imbalance ratio (ρ) is defined as the ratio of the number of samples in the majority class to the number of samples in the minority class. To evaluate the effectiveness of the proposed method at different imbalance steps, we executed our suggested approach on the MNIST [35] (a balanced dataset with 70,000 handwritten digit images, 60,000 for training and 10,000 for testing) and Cifar -10 datasets with varying numbers of minority classes and increasing imbalance ratios. For both datasets, we considered 2, 5, and 8 minority classes. Since the MNIST dataset was much simpler compared to Cifar -10, we considered imbalance ratios of {1, 500, 1000, 2000, 3000, 4000} for MNIST. However, achieving acceptable results in Cifar -10 with reduced data was more challenging. Therefore, we considered imbalance ratios of {1, 10, 20, 30, 40, 50} for Cifar -10.

To evaluate the performance of the method at different levels of imbalance, we used the area under the receiver operating characteristic curve (ROC AUC) metric, which is widely used to measure the performance of classifiers. Although the original version of ROC AUC was designed for binary classifiers, we utilized a multi-class modification of it. By treating each class as a positive class and the rest as negative classes (one vs. all), the ROC AUC provided a measure of classification performance for each specific class. The average of the ROC AUCs was then computed across all the binary classification problems.

As expected, increasing the imbalanced rate had a negative impact on the performance of the proposed method in both MNIST (Fig. 9) and Cifar-10 (Fig. 10). However, by comparing the curves (a-c) in Figs. 9 and 10, it can be seen that as the imbalance increased, the performance gap between the proposed DWFL method and the baseline model with cross-entropy loss function increased, as well. This indicates that the proposed method had better performance than cross-entropy in the high imbalance scenarios.

5.10 Investigating the effectiveness of dynamic weighting

To evaluate the effectiveness of dynamic weighting in this section, three distinct models that make use of the focal loss function were taken into account. Models with no weighing technique for their loss function were referred to as FL, those with constant weighing were referred to as WFL, and those with dynamic weighting were referred to as DWFL. The number used to the right of the model name denoted the gamma value, for example, DWFL2. For each of them, the CNN configuration was the same and resembles Table 2. We trained CNN with all three loss functions considering gamma = 2 on the Synthetic dataset introduced in Sect. 5.1. The accuracy of the models in each class is seen in Fig. 11. With 400 samples, the minority class C2 and majority class C3 both had the best results, which were connected to DWFL. In the other class C1, the results were also good and close to the best.

Table 4 Comparing the results obtained by CE, WCE, and DWFL loss functions with ResNet-18 on the manually imbalanced Cifar-10 dataset

	Train accuracy (%)	Test accuracy (%)	Balanced accuracy (%)	F1-score (%)
ResNet-18 (CE)	91.47	80.55	75.76	81.91
ResNet-18 + (weighted CE)	91.80	75.21	72.40	73.90
ResNet-18 + DWFL1	**92.56**	**82.**43	81.**87**	**82.67**

The best result in each column is indicated in bold and underlined format

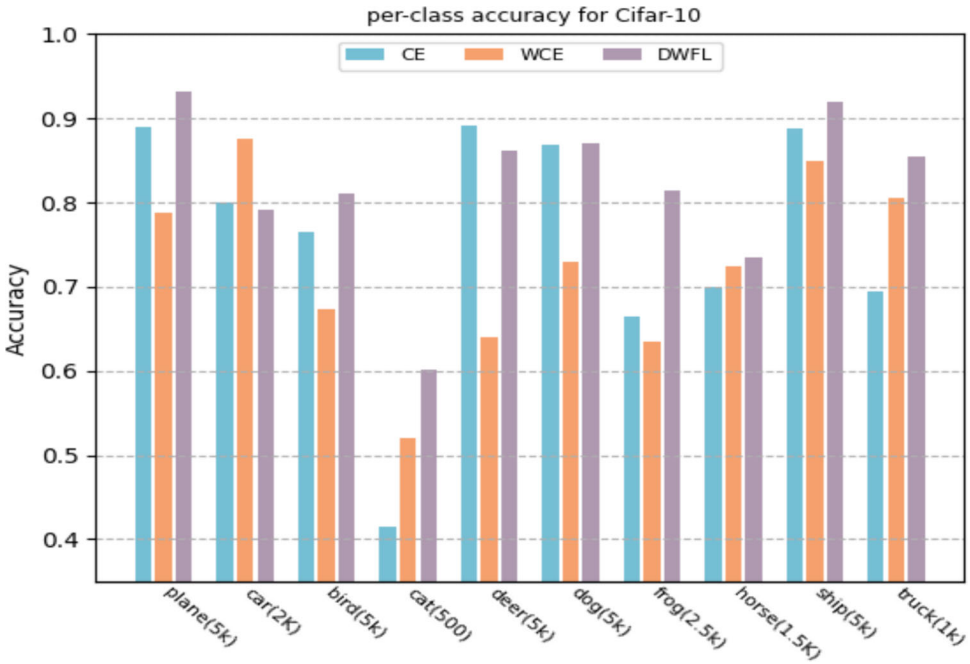

Fig. 6 The per-class accuracy of ResNet-18 trained with three different loss functions, Cross-Entropy (CE) loss function, Weighted Cross-Entropy (WCE) loss function, and our proposed Dynamic Weighted Focal Loss (DWFL) on the manually imbalanced cifar-10 dataset. The size of each class is written next to its name, with the class 'Cat' having a size of 500 and the class 'Ship' having a size of 5000. In both of them, our proposed method has the best performance

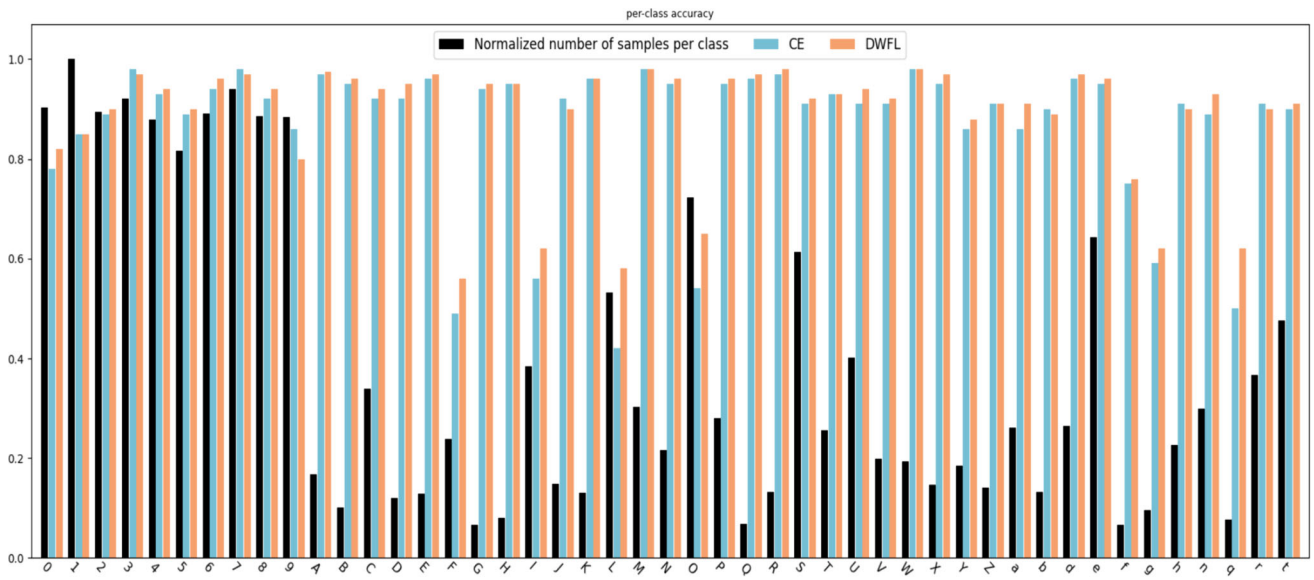

Fig. 7 Our proposed DWFL and CE testing accuracy for different classes on EMNIST-by merge. The black bars represent the normalized distribution of data in the 47 classes of the EMNIST-by merge dataset [25], where each class has a different number of examples, and it is an imbalanced dataset. As shown in the bar charts, the DWFL method achieves higher accuracy in most classes, both among the classes with limited data and among the classes with abundant data (color figure online)

Table 5 Comparing the results obtained by VGG5 with CE, WCE, and DWFL loss functions on the EMNIST-by merge dataset

	Train accuracy (%)	Test accuracy (%)	Balanced accuracy (%)	F1-score (%)
VGG5 (CE)	91.47	89.01	86.82	82.09
VGG5 + DWFL2	**92.56**	**90.89**	**88.87**	**83.56**

The best result in each column is indicated in bold and underlined format

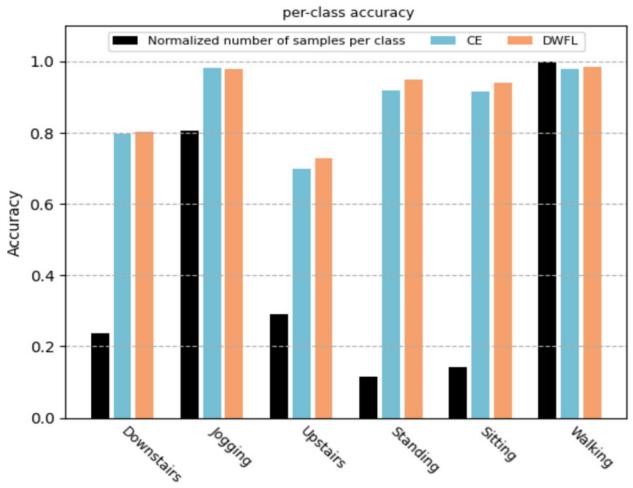

Fig. 8 Comparing testing accuracies for CNN with two loss functions: Cross-Entropy (CE) and our proposed Dynamic Weighted Focal Loss (DWFL) for different classes on the Actitracker dataset. The black bars represent the normalized distribution of data in the 6 classes of the actitracker dataset [31], where each class has a different number of examples, Which causes an imbalanced dataset. the proposed DWFL method demonstrates better performance both in minority classes like "standing" and in majority classes like "walking." (color figure online)

To further investigate the effectiveness of dynamic weighting, we plotted the changes in F1-score and accuracy relative to variations in gamma. A graph of the F1-score metric with different gamma values for three loss functions of standard focal loss (F-score), weighted focal loss (W–F-score), and dynamic weighted focal loss (DW-F-score), is shown in Fig. 12. Two points can be deduced from this graph. First, DWFL had the best performance by selecting gamma equal to 2 and 3. Second that DWFL was more stable than gamma changes, while FL and WFL performance decreased sharply with increasing gamma, which seems to be due to a sharp decline in the share of more samples in the final loss. Additionally, in Fig. 13, the curve depicting the changes in accuracy relative to variations in gamma has been plotted. Comparing the accuracy curves of FL and DWFL, it is evident that the proposed DWFL method is more robust to changes in gamma compared to FL. This robustness to an increase in gamma in Figs. 12 and 13 can be interpreted as follows: Although the focal loss function treats a significant number of samples as easy samples and disregards their role in the update process, leading to a decrease in classifier performance for certain classes, the dynamic weighting of DWFL compensates for this effect. By robustly adjusting the weights, DWFL demonstrates higher resistance to the increase in gamma.

6 Conclusion

In this paper, a new weighted loss function based on the focal loss was addressed to deal with class imbalance. This method determined the weight of samples based on two components, first, whether the sample is easy or hard, and second, how much the class is learned in the previous epochs. Section 4 details the convolutional neural network training algorithm based on our new loss function. This algorithm was proposed for weighing classes and determining the weight of each class based on the results of the previous epochs of training so that the well-learned classes were down-weighed and the bad-learned classes up-weighed. The performance of the proposed loss function was then evaluated on four datasets, including the Synthetic dataset, Cifar-10 dataset, EMNIST-by merge dataset, and Actitracker dataset. The details of the results were provided in Sect. 5. The proposed method has demonstrated high performance in addressing imbalanced data, as

Table 6 Topology of CNN proposed for the actitracker dataset

Layer	Number of filters/number of neurons	Kernel size
1D Convolution ReLU	100	10×1
Max-Pooling kernel	–	3×1
1D Convolution ReLU	160	10×1
Global-Average-Pooling Dropout 20%	–	–
Fully connected SoftMax	6 Neurons	–

Table 7 Comparison of the performance of Cross-Entropy (CE), Weighted Cross-Entropy (WCE), and our proposed DWFL on the Actitracker dataset

	Train accuracy (%)	Test accuracy (%)	Balanced accuracy (%)	F1-score (%)
CNN	93.73	92.25	88.14	87.17
CNN + (weighted CE)	92.62	91.42	87.04	86.75
CNN + DWFL2	**93.91**	**93.31**	**89.65**	**88.23**

The best result in each column is indicated in bold and underlined format

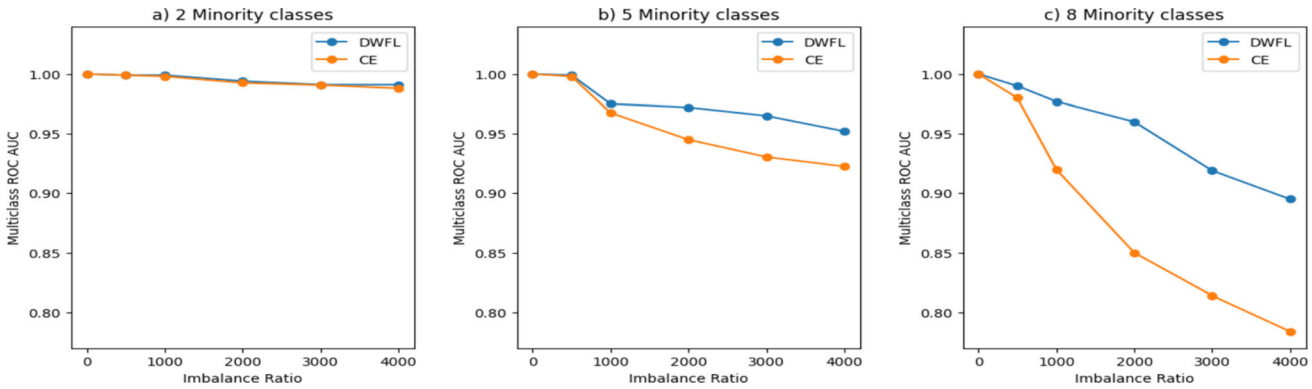

Fig. 9 The multi-class ROC AUCs for various imbalance ratio and fix number of minority class in the MNIST dataset

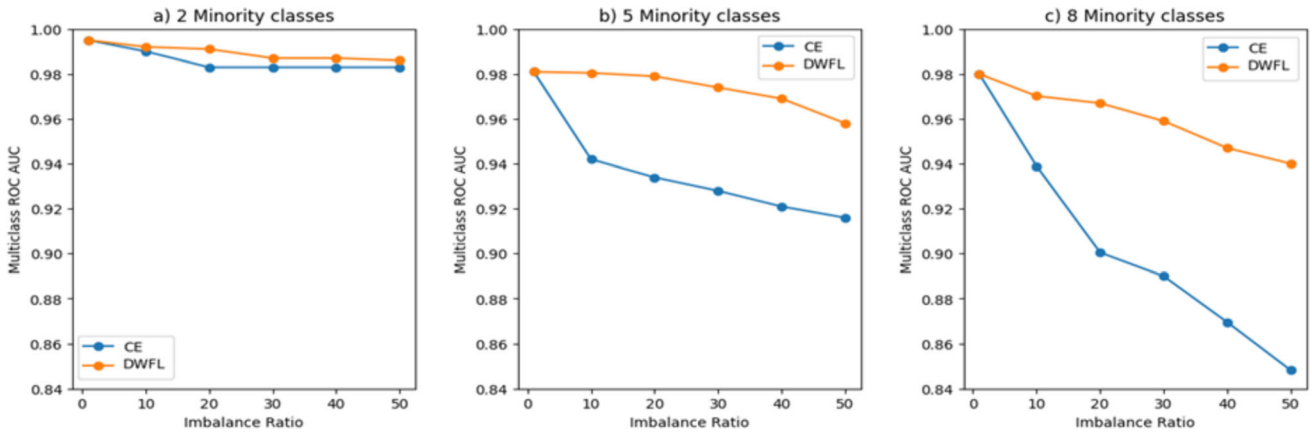

Fig. 10 The multi-class ROC AUCs for various imbalance ratio and fix number of minority class in the Cifar-10 dataset

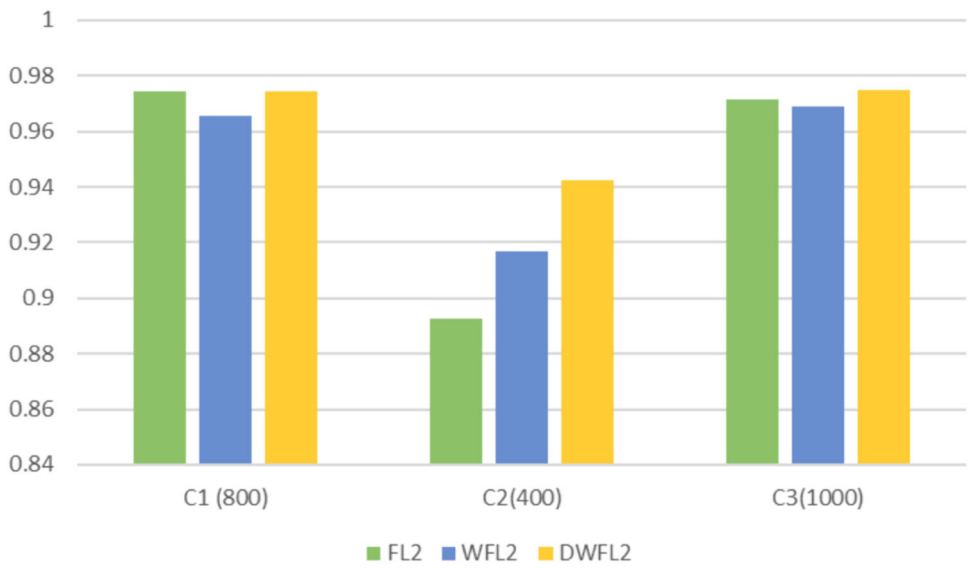

Fig. 11 To evaluate the performance of dynamic weight, the accuracy of three loss functions, namely Focal Loss (FL), weighted focal loss (WFL), and dynamic weighted focal loss (DWFL), in each class of the Synthetic dataset has been compared. The number 2 on the right side of the term "FL2" represents selecting gamma as 2

evidenced by the observed results on the evaluated datasets. It exhibits high accuracy across the most of classes. However, it is important to note that the method has its limitations. Firstly, the dynamic nature of class weights introduces computational overhead, leading to increased training time and resource requirements. Secondly, the performance of the loss function is sensitive to hyperparameter choices, necessitating careful tuning for optimal

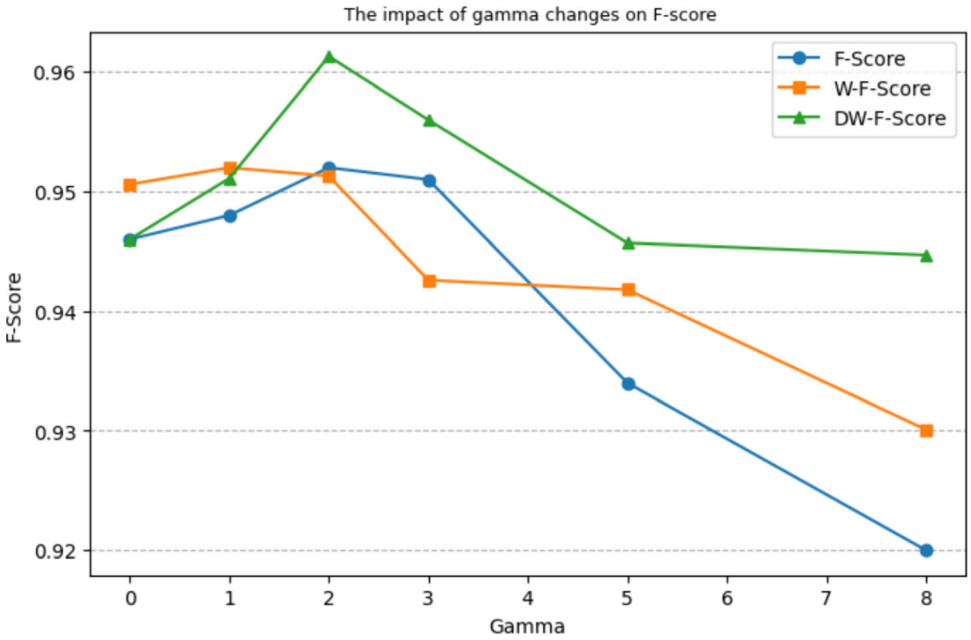

Fig. 12 Investigating the effect of gamma changes on F1-score in FL, WFL, and DWFL

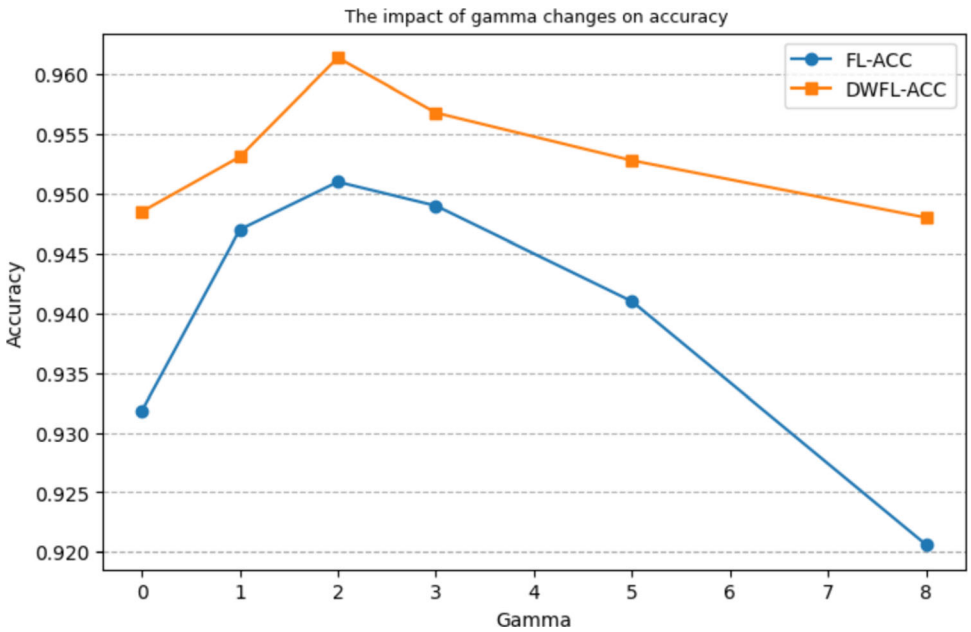

Fig. 13 Testing accuracies of our DWFL and FL for different values of gamma

results. Lastly, the reliance on validation data for determining class weights may result in varying effectiveness with different validation sets. More work can be done on measuring the effectiveness of the focal loss function for non-convolutional DNNs to investigate whether the methods presented can be applied to other types of architecture, e.g., RNNs and LSTM. We propose applying the DWFL to more complex and bigger datasets with varying degrees of imbalance in future work.

Funding Malek-Ashtar University of Technology.

Data availability Data sharing not applicable to this article as no datasets were generated or analyzed during the current study.

Declarations

Conflict of interest The authors declare no conflicts of interest.

References

1. Anand R, Mehrotra KG, Mohan CK, Ranka S (1993) An improved algorithm for neural network classification of

imbalanced training sets. IEEE Trans Neural Netw 4(6):962–969. https://doi.org/10.1109/72.286891
2. Johnson JM, Khoshgoftaar TM (2019) Survey on deep learning with class imbalance. J Big Data 6(1):27. https://doi.org/10.1186/s40537-019-0192-5
3. Masko D, Hensman P (2015) The impact of imbalanced training data for convolutional neural networks
4. Pouyanfar S et al (2018) Dynamic sampling in convolutional neural networks for imbalanced data classification. In: 2018 IEEE conference on multimedia information processing and retrieval (MIPR), 10–12 April 2018, pp 112–117. https://doi.org/10.1109/MIPR.2018.00027
5. Buda M, Maki A, Mazurowski MA (2018) A systematic study of the class imbalance problem in convolutional neural networks. Neural Netw 106:249–259. https://doi.org/10.1016/j.neunet.2018.07.011
6. He H, Bai Y, Garcia E, Li S (2008) ADASYN: adaptive synthetic sampling approach for imbalanced learning, pp 1322–1328
7. Wang S, Liu W, Wu J, Cao L, Meng Q, Kennedy PJ (2016) Training deep neural networks on imbalanced data sets. Piscataway NJ (Ed.) Institute of electrical and electronics engineers (IEEE), pp 4368–4374
8. Lin T, Goyal P, Girshick R, He K, Dollár P (2017) Focal loss for dense object detection. In: 2017 IEEE international conference on computer vision (ICCV), 22–29 Oct 2017, pp 2999–3007. https://doi.org/10.1109/ICCV.2017.324
9. Lin T-Y et al (2014) Microsoft COCO: common objects in context
10. Shrivastava A, Sukthankar R, Malik J, Gupta AK (2016) Beyond skip connections: top-down modulation for object detection. ArXiv, vol. abs/1612.06851
11. Fu C-Y, Liu W, Ranga A, Tyagi A, Berg AC (2017) DSSD: deconvolutional single shot detector, [Online]. Available: http://arXiv.org/abs/
12. Qiao Z, Bae A, Glass LM, Xiao C, Sun J (2020) FLANNEL (Focal loss based neural network ensembLe) for COVID-19 detection. J Am Med Inform Assoc 28(3):444–452. https://doi.org/10.1093/jamia/ocaa280
13. Gao F, Zhu J, Jiang H, Niu Z, Han W, Yu J (2020) Incremental focal loss GANs. Inf Process Manage 57(3):21. https://doi.org/10.1016/j.ipm.2019.102192
14. Isola P, Zhu J, Zhou T, Efros AA (2017) Image-to-image translation with conditional adversarial networks. In: 2017 IEEE conference on computer vision and pattern recognition (CVPR), 21–26 July 2017, pp 5967–5976. https://doi.org/10.1109/CVPR.2017.632
15. Ledig C et al (2017) Photo-realistic single image super-resolution using a generative adversarial network. In: 2017 IEEE conference on computer vision and pattern recognition (CVPR), 21–26 July 2017, pp 105–114. https://doi.org/10.1109/CVPR.2017.19
16. Zhang H, Goodfellow I, Metaxas D, Odena A (2019) Self-attention generative adversarial networks. In: Presented at the proceedings of the 36th international conference on machine learning, proceedings of machine learning research. [Online]. Available: https://proceedings.mlr.press/v97/zhang19d.html
17. Liu Z, Luo P, Wang X, Tang X (2015) Deep learning face attributes in the wild. In: 2015 IEEE international conference on computer vision (ICCV), pp 3730–3738
18. Yu F, Zhang Y, Song S, Seff A, Xiao J (2015) LSUN: construction of a large-scale image dataset using deep learning with humans in the loop. CoRR, vol. abs/1506.03365 [Online]. Available: http://arxiv.org/abs/1506.03365.
19. Krizhevsky A (2012) Learning multiple layers of features from tiny images. University of Toronto, 05/08 2012
20. Goodfellow IJ et al. (2014) Generative adversarial nets. In: Presented at the proceedings of the 27th international conference on neural information processing systems – Vol. 2, Montreal, Canada
21. Mao X, Li Q, Xie H, Lau RYK, Wang Z, Smolley SP (2017) Least squares generative adversarial networks. In: 2017 IEEE international conference on computer vision (ICCV), 22–29 Oct. 2017, pp 2813–2821. https://doi.org/10.1109/ICCV.2017.304
22. Lim JH, Ye JC (2017) Geometric GAN
23. Mukhoti J, Kulharia V, Sanyal A, Golodetz S, Torr PHS, Dokania PK (2020) Calibrating deep neural networks using focal loss. [Online]. Available: http://arXiv.org/abs/
24. Hastie TJ, Rosset S, Zhu J, Zou H (2009) Multi-class AdaBoostB. Stat Interface 2:349–360
25. Cohen G, Afshar S, Tapson J, Schaik AV (2017) EMNIST: an extension of MNIST to handwritten letters. [Online]. Available: http://arXiv.org/abs/
26. Kwapisz JR, Weiss GM, Moore SA (2011) Activity recognition using cell phone accelerometers. SIGKDD Explor Newsl 12(2):74–82. https://doi.org/10.1145/1964897.1964918
27. Guo B, Chen S, Hong Z, Xu G (2022) Pattern Recognition and Analysis: Neural Network using Weighted Cross Entropy. J Phys Conf Ser 2218(1):012043. https://doi.org/10.1088/1742-6596/2218/1/012043
28. Zhou Z, Huang H, Fang B (2021) Application of weighted cross-entropy loss function in intrusion detection. J Comput Commun 9(11):1–21
29. Chawla N, Bowyer K, Hall L, Kegelmeyer W (2002) SMOTE: synthetic minority over-sampling technique. J Artif Intell Res (JAIR) 16:321–357. https://doi.org/10.1613/jair.953
30. Joloudari JH, Marefat A, Nematollahi MA, Oyelere SS, Hussain S (2023) Effective class-imbalance learning based on SMOTE and convolutional neural networks. Appl Sci 13(6):4006
31. Taherkhani A, Cosma G, McGinnity TM (2020) AdaBoost-CNN: an adaptive boosting algorithm for convolutional neural networks to classify multi-class imbalanced datasets using transfer learning. Neurocomputing 404:351–366. https://doi.org/10.1016/j.neucom.2020.03.064
32. He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: 2016 IEEE conference on computer vision and pattern recognition (CVPR), 27–30 June 2016, pp 770–778. https://doi.org/10.1109/CVPR.2016.90.
33. Prajwal TS, AK I (2023) A comparative study Of RESNET-pretrained models for computer vision. In: Presented at the proceedings of the 2023 fifteenth international conference on contemporary computing, <conf-loc>, <city>Noida</city>, <country>India</country>, </conf-loc>. [Online]. Available: https://doi.org/10.1145/3607947.3608042
34. Kabir HMD et al (2020) SpinalNet: deep neural network with gradual input. [Online]. Available: http://arXiv.org/abs/
35. LeCun Y (1998) The MNIST database of handwritten digits. http://yann.lecun.com/exdb/mnist/